Experiment 'p_13_10_recursive_area_polygon_seq' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 4
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 4

Problem Specification

Problem name: p_13_10_recursive_area_polygon_seq
Natural language statement: null
Method signature: p_13_10_recursive_area_polygon_seq(points: seq<(real, real)>) returns (area: real)

Ensures

- `area == polygon_area(points)`

Requires

- `|points| >= 3`

Functional Code Given

```
function polygon_area(points: seq<(real, real)>): real
  decreases |points|
  requires |points| >= 3
  ensures polygon_area(points) >= 0.0
{
  if |points| == 3 then
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real
  else
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
```

```
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real;
    triangle_area + polygon_area([points[0]] + points[2..])
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_13_10_recursive_area_polygon_seq(points: seq<(real, real)>) returns (area: real)

The method should respect the following contract:

requires |points| >= 3, ensures area == polygon_area(points)

The contract uses the following dafny code:

function polygon_area(points: seq<(real, real)>): real
  decreases |points|
  requires |points| >= 3
  ensures polygon_area(points) >= 0.0
{
  if |points| == 3 then
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real
  else
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real;
    triangle_area + polygon_area([points[0]] + points[2..])
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnPHTPShQ4RYY6aUWsm0rG57Xlv5C
Model: gpt-5.5-2026-04-23

Created at: 1780667623
Finish reason: stop
Usage: 896 tokens in, and 1367 tokens out

```
{
  if |points| == 3 {
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;
  } else {
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

    var remaining := [points[0]] + points[2..];
    assert |remaining| == |points| - 1;
    assert |remaining| >= 3;

    var rest := p_13_10_recursive_area_polygon_seq(remaining);
    area := triangle_area + rest;
  }
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_13_10_recursive_area_polygon_seq/program.dfy(24,0): Error: a postcondition
    ↪  could not be proved on this return path
   |
24 | {
   | ^

problem_data/p_13_10_recursive_area_polygon_seq/program.dfy(23,14): Related location:
    ↪ this is the postcondition that could not be proved
   |
23 |  ensures area == polygon_area(points)
   |  ^^

problem_data/p_13_10_recursive_area_polygon_seq/program.dfy(42,50): Error: cannot prove
    ↪ termination; try supplying a decreases clause
   |
42 | var rest := p_13_10_recursive_area_polygon_seq(remaining);
   | ^


Dafny program verifier finished with 2 verified, 2 errors
```

```
Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnPILT6Cf7mNf4JCk1Geqb0j6Jgs7
Model: gpt-5.5-2026-04-23
Created at: 1780667677
Finish reason: stop
Usage: 1270 tokens in, and 3541 tokens out

```
{
  var acc: real := 0.0;
  var q := points;

  while |q| > 3
    invariant |q| >= 3
    invariant acc + polygon_area(q) == polygon_area(points)
    decreases |q|
  {
    var (x1, y1) := q[0];
    var (x2, y2) := q[1];
    var (x3, y3) := q[2];
    var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

    var qNext := [q[0]] + q[2..];
    assert |qNext| == |q| - 1;
    assert |qNext| >= 3;
    assert polygon_area(q) == triangle_area + polygon_area(qNext);

    acc := acc + triangle_area;
    q := qNext;
  }

  assert |q| == 3;
  var (x1, y1) := q[0];
  var (x2, y2) := q[1];
  var (x3, y3) := q[2];
  var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
  var triangle_area: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;
  assert polygon_area(q) == triangle_area;

  area := acc + triangle_area;
  assert area == polygon_area(points);
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_13_10_recursive_area_polygon_seq/program.dfy(42,27): Error: assertion
    ↪ might not hold
   |
42 | assert polygon_area(q) == triangle_area + polygon_area(qNext);
   | ^^


Dafny program verifier finished with 2 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnPK5KTNdS5kUiYwmGFwhz00VAAFE
Model: gpt-5.5-2026-04-23
Created at: 1780667785
Finish reason: stop
Usage: 1745 tokens in, and 4051 tokens out

```
{
  var acc: real := 0.0;
  var q := points;
  assert q == points;
  assert acc + polygon_area(q) == polygon_area(points);

  while |q| > 3
    invariant |q| >= 3
    invariant acc + polygon_area(q) == polygon_area(points)
    decreases |q|
  {
    var (x1, y1) := q[0];
    var (x2, y2) := q[1];
    var (x3, y3) := q[2];
    var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

    var qNext := [q[0]] + q[2..];
    assert 4 <= |q|;
    assert |q[2..]| == |q| - 2;
    assert |qNext| == |q| - 1;
    assert |qNext| >= 3;
    assert qNext == [q[0]] + q[2..];
    assert q[0] == (x1, y1);
    assert q[1] == (x2, y2);
    assert q[2] == (x3, y3);
```

```
  calc {
    polygon_area(q);
  == {
    assert |q| != 3;
    assert q[0] == (x1, y1);
    assert q[1] == (x2, y2);
    assert q[2] == (x3, y3);
  }
    triangle_area + polygon_area([q[0]] + q[2..]);
  == {
    assert qNext == [q[0]] + q[2..];
  }
    triangle_area + polygon_area(qNext);
  }

  calc {
    (acc + triangle_area) + polygon_area(qNext);
  == {
  }
    acc + (triangle_area + polygon_area(qNext));
  == {
    assert polygon_area(q) == triangle_area + polygon_area(qNext);
  }
    acc + polygon_area(q);
  == {
    assert acc + polygon_area(q) == polygon_area(points);
  }
    polygon_area(points);
  }

  acc := acc + triangle_area;
  q := qNext;
}

assert |q| == 3;
var (x1, y1) := q[0];
var (x2, y2) := q[1];
var (x3, y3) := q[2];
var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
var triangle_area: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;
assert q[0] == (x1, y1);
assert q[1] == (x2, y2);
assert q[2] == (x3, y3);

calc {
  polygon_area(q);
== {
  assert |q| == 3;
  assert q[0] == (x1, y1);
  assert q[1] == (x2, y2);
```

```
    assert q[2] == (x3, y3);
  }
    triangle_area;
  }

  area := acc + triangle_area;
  calc {
    area;
  == {
  }
    acc + triangle_area;
  == {
    assert polygon_area(q) == triangle_area;
  }
    acc + polygon_area(q);
  == {
    assert acc + polygon_area(q) == polygon_area(points);
  }
    polygon_area(points);
  }
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_13_10_recursive_area_polygon_seq/program.dfy(59,20): Error: the
    ↪ calculation step between the previous line and this line could not be proved
   |
59 | triangle_area + polygon_area([q[0]] + q[2..]);
   |  ^


Dafny program verifier finished with 2 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnPLnw7HDd7LUiurYOJapoCNm187L
Model: gpt-5.5-2026-04-23
Created at: 1780667891
Finish reason: stop
Usage: 2853 tokens in, and 7905 tokens out

```
{
  var acc: real := 0.0;
  var q := points;
```

```
  assert acc + polygon_area(q) == polygon_area(points);

  while |q| > 3
    invariant |q| >= 3
    invariant acc + polygon_area(q) == polygon_area(points)
    decreases |q|
  {
    var sign: real := (q[0]).0 * ((q[1]).1 - (q[2]).1) + (q[1]).0 * ((q[2]).1 - (q[0]).1)
        ↪ + (q[2]).0 * ((q[0]).1 - (q[1]).1);
    var triangle_area: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

    var qNext := [q[0]] + q[2..];
    assert |q[2..]| == |q| - 2;
    assert |qNext| == |q| - 1;
    assert |qNext| >= 3;
    assert qNext == [q[0]] + q[2..];

    assert polygon_area(q) == triangle_area + polygon_area(qNext) by {
      reveal polygon_area();
      assert |q| != 3;
      assert qNext == [q[0]] + q[2..];
    }

    var newAcc := acc + triangle_area;
    assert newAcc + polygon_area(qNext) == acc + (triangle_area + polygon_area(qNext));
    assert newAcc + polygon_area(qNext) == acc + polygon_area(q);
    assert newAcc + polygon_area(qNext) == polygon_area(points);

    acc := newAcc;
    q := qNext;
  }

  assert |q| == 3;
  var sign: real := (q[0]).0 * ((q[1]).1 - (q[2]).1) + (q[1]).0 * ((q[2]).1 - (q[0]).1) +
      ↪  (q[2]).0 * ((q[0]).1 - (q[1]).1);
  var triangle_area: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

  assert polygon_area(q) == triangle_area by {
    reveal polygon_area();
    assert |q| == 3;
  }

  area := acc + triangle_area;
  assert area == acc + polygon_area(q);
  assert area == polygon_area(points);
}
```

Final Program

```
function polygon_area(points: seq<(real, real)>): real
```

```
  decreases |points|
  requires |points| >= 3
  ensures polygon_area(points) >= 0.0
{
  if |points| == 3 then
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real
  else
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real;
    triangle_area + polygon_area([points[0]] + points[2..])
}

method p_13_10_recursive_area_polygon_seq(points: seq<(real, real)>) returns (area: real)
       requires |points| >= 3
       ensures area == polygon_area(points)
{
  var acc: real := 0.0;
  var q := points;
  assert acc + polygon_area(q) == polygon_area(points);

  while |q| > 3
    invariant |q| >= 3
    invariant acc + polygon_area(q) == polygon_area(points)
    decreases |q|
  {
    var sign: real := (q[0]).0 * ((q[1]).1 - (q[2]).1) + (q[1]).0 * ((q[2]).1 - (q[0]).1)
        ↪ + (q[2]).0 * ((q[0]).1 - (q[1]).1);
    var triangle_area: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

    var qNext := [q[0]] + q[2..];
    assert |q[2..]| == |q| - 2;
    assert |qNext| == |q| - 1;
    assert |qNext| >= 3;
    assert qNext == [q[0]] + q[2..];

    assert polygon_area(q) == triangle_area + polygon_area(qNext) by {
      reveal polygon_area();
      assert |q| != 3;
      assert qNext == [q[0]] + q[2..];
    }

    var newAcc := acc + triangle_area;
    assert newAcc + polygon_area(qNext) == acc + (triangle_area + polygon_area(qNext));
    assert newAcc + polygon_area(qNext) == acc + polygon_area(q);
```

```
    assert newAcc + polygon_area(qNext) == polygon_area(points);

    acc := newAcc;
    q := qNext;
  }

  assert |q| == 3;
  var sign: real := (q[0]).0 * ((q[1]).1 - (q[2]).1) + (q[1]).0 * ((q[2]).1 - (q[0]).1) +
      ↪  (q[2]).0 * ((q[0]).1 - (q[1]).1);
  var triangle_area: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

  assert polygon_area(q) == triangle_area by {
    reveal polygon_area();
    assert |q| == 3;
  }

  area := acc + triangle_area;
  assert area == acc + polygon_area(q);
  assert area == polygon_area(points);
}
```

Total Token Usage

Input tokens: 6764
Output tokens: 16864
Reasoning tokens: 14386
Sum of 'total tokens': 23628

Experiment Timings

Overall Experiment started at 1780667623432, ended at 1780668070384, lasting 446952ms (446.95 seconds)
Iteration #4 started at 1780667890319, ended at 1780668070384, lasting 180065ms (180.07 seconds)
Iteration #1 started at 1780667623433, ended at 1780667676577, lasting 53144ms (53.14 seconds)
Iteration #2 started at 1780667676580, ended at 1780667784433, lasting 107853ms (107.85 seconds)
Iteration #3 started at 1780667784433, ended at 1780667890298, lasting 105865ms (105.87 seconds)